

MONTREAL.AI RESEARCH

GoalOS V97.4

Private On-Chain Validated Skill Graph

*ENS-Gated Agent Writes, Encrypted Skill Capsules, and Zero-Knowledge
Chronicle Authority*

Design law

GoalOS V97.4 secures the Validated Skill Graph as Ethereum mainnet proof commitments and optional encrypted on-chain capsules. Direct AGI Agents - wallets that own `*.agent.agi.eth` - may write, supersede, or revoke skill commitments. Private intelligence is never written as plaintext. If it must live on-chain, it is written as ciphertext plus commitments; zero-knowledge proofs attest policy satisfaction without revealing the capsule.

Claim boundary. This paper defines a research and implementation architecture. It does not claim achieved AGI, achieved ASI, empirical state of the art, legal certification, financial advice, live settlement, or production authority. Ethereum mainnet deployment, cryptography, smart contracts, zk circuits, key management, and validator operations require independent review.

Abstract

Frontier models make candidate output abundant. Institutions need something different: proof-carrying memory that can survive replay, validation, scope checks, rollback, and adversarial review. This paper specifies **GoalOS V97.4**, a mainnet-secured architecture for the VALIDATED SKILL GRAPH. The architecture stores public proof commitments, evidence roots, ProofBundle roots, schema hashes, version events, and authorization records on Ethereum mainnet. It restricts graph updates to wallets that own direct `*.agent.agi.eth` subnames, such as `john.agent.agi.eth`, using ENS Registry and NameWrapper ownership checks. It adds a privacy-preserving storage model in which private skill intelligence can be kept as encrypted ciphertext on-chain or off-chain, while zero-knowledge proofs attest hidden policy predicates such as proof-level satisfaction, validator quorum, replay availability, blocked-claim exclusion, and Chronicle eligibility. The result is a proof-governed memory substrate: agents may write only under verifiable identity authority, validators may attest without exposing private skill contents, and future missions may rely only on Chronicle-admitted capability.

Executive summary

The durable unit is not a prompt or transcript. It is a validated skill: a scoped, replayable, evidence-backed capability admitted through a policy gate and safe to reuse. V97.4 adds an Ethereum mainnet security layer: direct AGI Agent authorization, encrypted private capsule handling, zero-knowledge policy proofs, and append-auditable commitment events.

1 Design Objective

GoalOS converts autonomous AI output into proof-bearing machine labor. The core loop is:

Objective → Mission Contract → Proof Debt → AGI Jobs → ProofBundles
→ Evidence Docket → Chronicle → Validated Skill.

The Validated Skill Graph is the memory layer of this loop. It should not admit raw language-model output. It should admit capability manifests only when evidence, replay, validation, risk, scope, and rollback conditions pass.

V97.4 addresses a deployment question: how can this graph be secured on Ethereum mainnet without leaking private operational intelligence? The answer is a layered design:

1. direct ENS-based write authorization for AGI Agents;
2. on-chain commitments and optional on-chain encrypted capsules;
3. zero-knowledge proofs for hidden policy predicates;
4. EIP-712 typed attestations for validators and agents;
5. Chronicle gates that convert proof into bounded future mission authority.

2 From Private Intelligence to Verifiable Mainnet Memory

The phrase “store intelligence on-chain” is dangerous if it means plaintext. Ethereum is public, durable, replicated, and adversarially readable. GoalOS therefore distinguishes four storage modes.

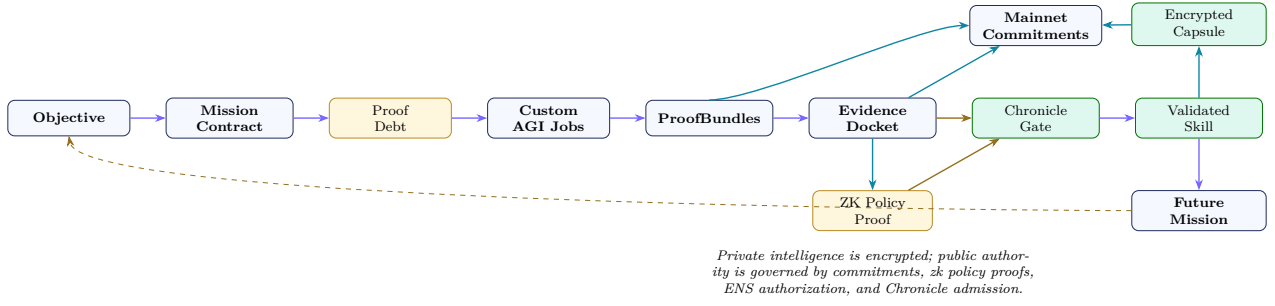


Figure 1. GoalOS V97.4 mainnet privacy architecture. Ethereum stores commitments, optional ciphertext capsules, authorization events, and zk proof verification results; Chronicle memory remains proof-gated.

Mode	On-chain material	Use case
Commitment-only	Graph root, Evidence Docket root, ProofBundle root, schema hash, version, agent node.	Default mode for sensitive or high-volume intelligence.
Encrypted URI	Commitment roots plus pointer to encrypted content-addressed payload.	Practical private graph storage with cheap on-chain verification.
Encrypted on-chain capsule	Compact ciphertext stored in contract storage, calldata, or data-availability layer, plus commitments.	High-value compact records requiring public availability of ciphertext.
ZK-verified hidden capsule	Public inputs and proof verify policy predicates over a hidden witness.	Prove eligibility without revealing method, evidence, or reviewer notes.

Table 1. Storage modes. V97.4 permits encrypted private intelligence on-chain, but never as plaintext; the recommended default is commitment-only or encrypted URI with zk policy proofs.

Privacy correction

A public blockchain cannot keep plaintext private. GoalOS can keep private intelligence *securely on-chain* only as ciphertext, commitments, and proofs. Long-term confidentiality then depends on encryption choices, key management, access control, revocation policy, and cryptographic assumptions. Highly sensitive raw content should remain encrypted off-chain with on-chain roots and zk proofs.

3 Direct AGI Agent Authorization

Write access is restricted to wallets that own a direct subname under `agent.agi.eth`. For a label λ , such as `john`, define:

$$\text{labelhash}(\lambda) = \text{keccak256}(\lambda), \quad \text{agentNode}(\lambda) = \text{namehash}(\lambda.\text{agent.agi.eth}).$$

Equivalently, if $P = \text{namehash}(\text{agent.agi.eth})$, the direct child node is

$$N_\lambda = \text{keccak256}(P, \text{labelhash}(\lambda)).$$

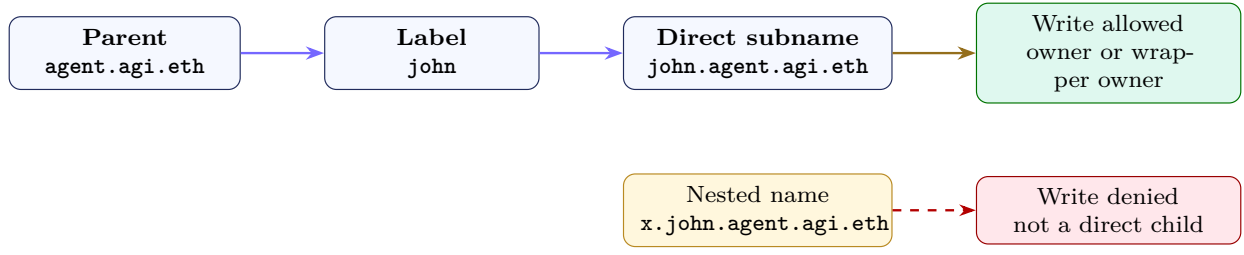
The registry accepts a write only if either:

$$\text{ENS.owner}(N_\lambda) = \text{msg.sender},$$

or, when the direct subname is wrapped,

$$\text{ENS.owner}(N_\lambda) = \text{NameWrapper} \quad \wedge \quad \text{NameWrapper.ownerOf}(N_\lambda) = \text{msg.sender}.$$

This rule authorizes `john.agent.agi.eth` but not `x.john.agent.agi.eth`. The latter is a deeper descendant, not a direct child of `agent.agi.eth`.



Only the direct child node $\text{namehash}(\lambda.\text{agent.agi.eth})$ grants write authority. Deeper names may exist, but they do not inherit registry update rights.

Figure 2. Direct AGI Agent authorization. Only wallets controlling $\lambda.\text{agent.agi.eth}$ may update graph commitments for label λ .

4 On-Chain Object Model

A skill capsule has a private and a public face. The private face contains method, reviewer notes, failure modes, model/tool traces, and sensitive operational context. The public face contains roots, hashes, pointers, proof levels, agent node, version, and events.

Validated Skill Capsule

A private capsule C contains the complete skill intelligence:

$$C = \langle id, type, scope, method, evidenceSet, proofLevel, validators, replay, risk, limits, utility, rollback, version \rangle.$$

Its public mainnet face $\Pi(C)$ contains only commitments and verification handles:

$$\Pi(C) = \langle agentNode, graphRoot, evidenceRoot, proofBundleRoot, schemaHash, proofLevel, version, uriSet, zkPolicyProof \rangle.$$

The main commitment is:

$$\text{skillCommitment} = H(\text{chainId}, \text{registry}, \text{agentNode}, \text{graphRoot}, \text{evidenceRoot}, \text{proofBundleRoot}, \text{schemaHash}, \text{version}).$$

The graph root may be a Merkle root over canonicalized capsule fields; the evidence root may be a Merkle root over Evidence Docket claims, blocked claims, reviewer reports, and replay artifacts; the ProofBundle root may commit to returned work packages and validator verdicts.

5 Zero-Knowledge Policy Proofs

Zero-knowledge is useful when the system needs to prove that a hidden skill capsule satisfies a policy without revealing the underlying intelligence. V97.4 defines a ZK POLICY PROOF over a private witness:

$$W = \langle C, E, P, V, K \rangle,$$

where C is the skill capsule, E the Evidence Docket, P the ProofBundle set, V the validator attestations, and K the encryption/key policy. Public inputs include the roots and policy identifier:

$$X = \langle graphRoot, evidenceRoot, proofBundleRoot, schemaHash, proofLevel, agentNode, policyHash \rangle.$$

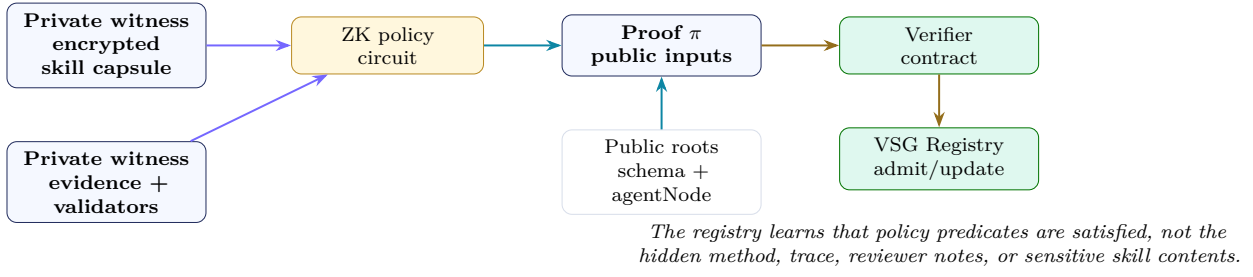


Figure 3. Zero-knowledge policy proof. Hidden skill intelligence remains private; public contracts verify eligibility predicates over commitments.

The proof statement is:

$$\text{Verify}_{zk}(X, \pi) = 1 \Rightarrow \left\{ \begin{array}{l} \text{capsule commits to } graphRoot, \\ \text{evidence commits to } evidenceRoot, \\ \text{ProofBundles commit to } proofBundleRoot, \\ \text{validator quorum is satisfied,} \\ \text{blocked claims are excluded,} \\ \text{proof level threshold is met,} \\ \text{rollback condition exists.} \end{array} \right.$$

6 Validator Attestations

Validators and AGI Agents should sign typed statements rather than ambiguous strings. V97.4 uses an EIP-712 style typed object:

```

SkillAttestation {
  bytes32 agentNode;
  bytes32 skillCommitment;
  bytes32 graphRoot;
  bytes32 evidenceRoot;
  bytes32 proofBundleRoot;
  uint8 proofLevel;
  bytes32 policyHash;
  uint64 expiresAt;
  string verdict; // supported | blocked | needs-more-proof
}

```

The attestation root becomes part of the ZK witness or Evidence Docket root. This lets validators attest privately, publicly, or selectively depending on the proof level.

7 Registry Surface

The minimal registry exposes three authority-changing operations:

```

registerSkill(labelhash, graphRoot, graphURI,
             evidenceRoot, evidenceURI,
             proofBundleRoot, proofBundleURI,
             proofLevel, schemaHash, zkProof);

supersedeSkill(oldSkillId, newRoots, newURIs, proofLevel,
               schemaHash, zkProof);

revokeSkill(skillId, reasonURI);

```

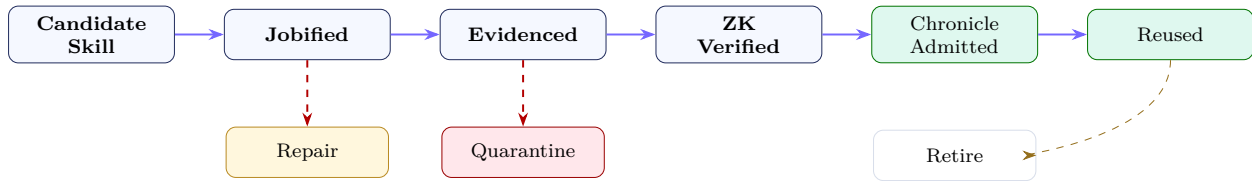


Figure 4. Encrypted and zk-verified lifecycle. Mainnet authority appears only after authorization, evidence, zk policy verification, and Chronicle admission.

The caller must own the direct AGI Agent subname for the label. The registry emits append-auditable events and never erases provenance. Supersession and revocation preserve history while removing or replacing future influence.

8 Security Invariants

The system is designed around security invariants rather than interface promises.

Invariant	Meaning
Direct-agent write authority	Only the owner of <code>λ.agent.agi.eth</code> may mutate records for <code>λ</code> .
No plaintext intelligence	Sensitive method, trace, reviewer notes, and domain evidence are never stored on-chain as plaintext.
Commitment consistency	On-chain roots must match canonicalized encrypted/off-chain payloads.
ZK-gated hidden claims	Hidden skill properties may grant authority only through verified policy proofs.
Chronicle non-contamination	Unproven or out-of-scope skills cannot influence future missions.
Supersession over deletion	Invalidated skills are superseded, revoked, or retired without erasing provenance.

Table 2. Security invariants for the encrypted mainnet Validated Skill Graph.

9 Threat Model

10 Implementation Checklist

A mainnet launch should not be treated as a UI deployment. It is a cryptographic and institutional system deployment.

1. Verify ownership and governance of `agent.agi.eth`.
2. Deploy or audit the VSG Registry contract.
3. Confirm ENS Registry and NameWrapper addresses for the target chain.
4. Canonicalize the skill-capsule JSON schema.
5. Choose storage mode: commitment-only, encrypted URI, encrypted on-chain capsule, or zk-verified hidden capsule.
6. Build test vectors for labelhash, namehash, agentNode, roots, and schema hashes.
7. Audit ZK circuits for proof-level, quorum, blocked-claim, replay, and rollback predicates.
8. Define EIP-712 attestation domain, typed data, expiry, and revocation policy.

Threat	Failure mode	Control
Name spoofing	A wallet claims to be an AGI Agent without owning the direct subname.	ENS Registry and NameWrapper owner checks.
Nested-subname escalation	A deeper subname tries to inherit root authority.	Direct-subname node computation only.
Ciphertext leakage	Encrypted intelligence is copied forever from public chain data.	Minimize on-chain ciphertext; use strong encryption and key governance.
Weak proof circuit	ZK proof omits a required predicate.	Circuit audits, public policy hash, test vectors, formal acceptance tests.
Validator collusion	Attesters approve weak skills.	Quorum, reputation, challenge windows, delayed outcome checks.
Memory contamination	Unsupported skill enters future missions.	Chronicle gate and non-contamination invariant.
Key compromise	Decryption keys leak.	Rotation, threshold access, revocation, supersession, and encrypted capsule retirement.

Table 3. Threats and controls. The central risk is not just data leakage; it is unauthorized authority over future work.

9. Run adversarial tests: spoofed subname, nested subname, wrong owner, stale proof, bad capsule, invalid proof, revoked skill.
10. Publish deployment address, ABI, schema hash, policy hash, and reviewer guide.

11 Evaluation Program

V97.4 should be evaluated under three categories.

Authorization tests. A direct owner of `john.agent.agi.eth` can register; a non-owner cannot; a wrapper owner can register; a nested subname cannot inherit authority.

Privacy tests. Plaintext method and reviewer notes do not appear on-chain. Ciphertext capsules decrypt only for authorized recipients. ZK proofs verify policy predicates while hiding witness fields.

Capability tests. A skill commitment cannot influence future missions until Chronicle admission is present. Superseded or revoked skills are excluded from reuse. Proof levels change validator burden and accepted evidence thresholds.

12 Claim Boundary

This paper does not claim that encrypted on-chain storage is automatically safe. Public-chain ciphertext is permanent and subject to future cryptanalysis and key compromise. For high-sensitivity intelligence, the preferred pattern is encrypted off-chain storage plus on-chain commitments and zk policy proofs. The encrypted on-chain capsule mode is reserved for compact, carefully classified records whose long-term exposure risk has been accepted by governance.

The paper also does not claim achieved AGI, achieved ASI, empirical state of the art, legal certification, financial advice, guaranteed ROI, live settlement, wallet safety, or production authority.

13 Conclusion

GoalOS V97.4 turns the Validated Skill Graph into an authorized, privacy-preserving mainnet memory substrate. Direct AGI Agents write. Independent validators attest. Ethereum stores commitments, optional ciphertext capsules, schema hashes, policy proofs, and version events. Zero-knowledge proofs let the system verify that hidden skill intelligence satisfies policy without exposing the intelligence itself. Chronicle gates preserve the central law: future work may rely only on capability that was proven, authorized, privacy-preserved, and safe to reuse.

Final line

The graph should not remember what was merely said. It should remember only what was proven, authorized, privacy-preserved, and safe to reuse.

References

- [1] Montreal.AI Research. *GoalOS: The Validated Skill Graph*. Research edition, 2026.
- [2] Montreal.AI Research. *GoalOS V85: A Unified Sovereign Proof Economy Operating System*. Research draft, 2026.
- [3] ENS Documentation. *Name Processing: namehash and labelhash*. <https://docs.ens.domains/resolution/names/>
- [4] ENS Documentation. *Deployments and mainnet registry*. <https://docs.ens.domains/learn/deployments/>
- [5] ENS Documentation. *Name Wrapper overview*. <https://docs.ens.domains/wrapper/overview/>
- [6] Ethereum Improvement Proposal 712. *Typed structured data hashing and signing*. <https://eips.ethereum.org/EIPS/eip-712>
- [7] Jens Groth. *On the Size of Pairing-Based Non-interactive Arguments*. EUROCRYPT, 2016.
- [8] Eli Ben-Sasson et al. *Zerocash: Decentralized Anonymous Payments from Bitcoin*. IEEE S&P, 2014.